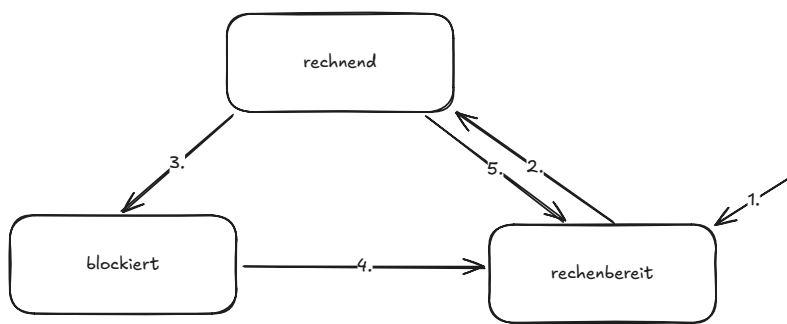


all



- 1.: neu erzeugter Prozess
2.: Dem Prozess wird die CPU zugeteilt
3.: Synchroner Unterbrechung: z.B. I/O Aufrufe: read, write...
4.: I/O ist beendet
5.: asynchrone Unterbrechung, externe Unterbrechung (z.B. Ablauf der Zeitscheibe)

Abnormales Prozessverhalten

Abnormales Prozessverhalten

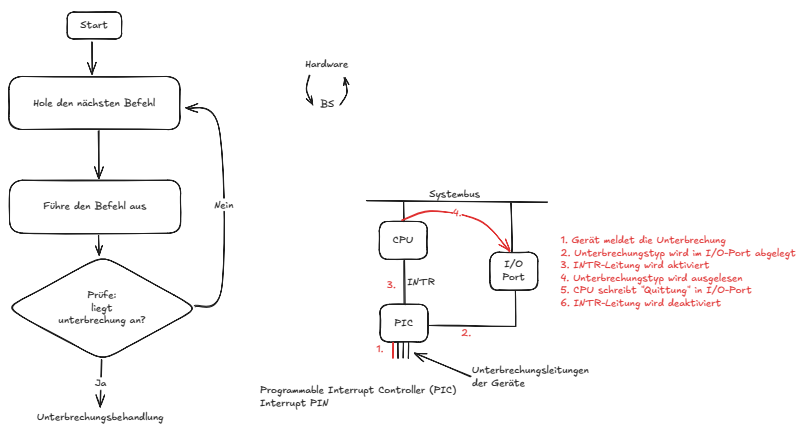
- **Deadlock:** Der Prozess ist blockiert und es erfolgt kein Zustandswechsel mehr.
- **Livelock:** Der Prozess wechselt ständig zwischen Zuständen, ohne in seiner Programmausführung voranzukommen.
- **Starvation:** Der Prozess bleibt rechenbereit, erhält aber keine Rechenzeit und kann daher nicht fortfahren

Ablauf Koordination

Ein BS hat eine Vielzahl von nebenläufigen Aktivitäten zu kontrollieren

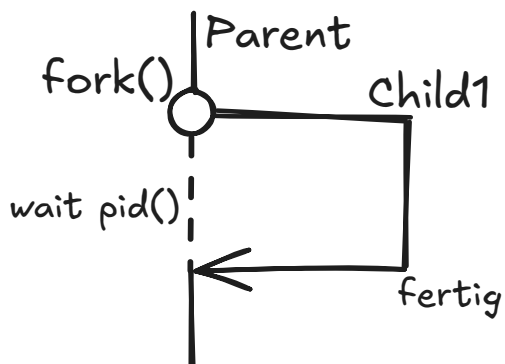
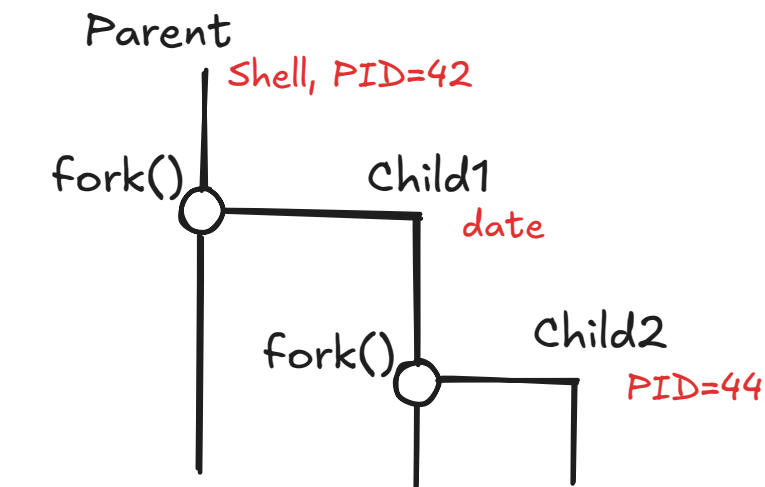
1. Polling := Aktiv reihum sich um die verschiedenen Aktivitäten kümmern (NIC, Benutzerprozesse, Plattencontroller, ...)
Vorteil: Fair
Nachteil: CPU-Zeit verschwendung wenn keine Aktivität vom BS erforderlich ist => Overhead
2. Unterbrechung

Abreitsweise der CPU



Prozess	Zustand initial	1	2	3	4
A	rechnend	rechnend	rechnend	rechenbereit	rechenbere
B	blockiert	blockiert	blockiert	blockiert	rechenbere

PCB: Process Control Block



Ablauf von fork()

1 Neuen Processblock

- Teil der Einträge werden neu gesetzt:
PID, PPI, ...
- Teil der Einträge werden auf Null gesetzt:
CPU-Verbrauch, Timer,
- Teil der Einträge werden vom Elternprocess kopiert:
Zeiger auf offene Dateideskriptoren, Prozessgruppe,
Signalzustand, Nice-Wert,...

2

3

Prozesskontrollblock

! Descriptor !

Copy on write

Bemerkung 1:

Seit 4.4 BSD effizientere implementierung von Schritt 2 im fork: Copy on Write

Die Seiten im virtuellen Adressraum als "copy on write" markiert, falls einer der beiden Prozesse eine Seite im Virtuellen Adressraum modifizieren will => Kopieren

$$fork = \begin{cases} 0 & \text{im Child} \\ \text{Child-PID} & \text{im Parent} \end{cases}$$

Parallel

Ctrl-Z $\hat{=}$ SIGTSTP

fg $\hat{=}$ SIGCONT

Ctrl-C $\hat{=}$ SIGKILL

